

MILTRACK

Military Asset Management System

System Documentation

Architecture · Database Schema · API Reference · RBAC Matrix · Setup Guide

Stack: React 18 (Vite) / Express.js / PostgreSQL / JWT Auth

1. Project Overview

MILTRACK is an enterprise-grade system for tracking military assets — vehicles, weapons, and ammunition — across multiple bases. It provides real-time inventory visibility, role-based access control (RBAC), atomic cross-base transfers, and a complete audit trail for every mutating action in the system.

Core Inventory Model

$$\text{Closing Balance} = \text{Opening Balance} + \text{Net Movement} - \text{Assigned} - \text{Expended}$$
$$\text{Net Movement} = \text{Purchases} + \text{Transfers In} - \text{Transfers Out}$$

Both balances are computed dynamically via SQL (Common Table Expressions) at query time rather than stored redundantly — this avoids data drift and keeps every figure traceable back to the underlying purchase, transfer, assignment, and expenditure records.

Tech Stack

Layer	Technology
Frontend	React 18, Vite, Recharts, Lucide React, Axios
Backend	Node.js, Express.js (ES Modules)
Database	PostgreSQL — raw SQL via the pg package (no ORM)
Authentication	JWT (jsonwebtoken), delivered via an httpOnly secure cookie
Security	Helmet, CORS (credentials allowlist), Express trust proxy, RBAC middleware
Deployment	Frontend -> Vercel/Netlify · Backend + DB -> Render/Railway or Neon/Supabase

2. System Architecture

The system follows a classic 3-tier architecture. The React frontend never talks to the database directly — every request goes through the Express API, which authenticates the caller, applies role-based authorization, executes parameterized SQL against PostgreSQL, and (for mutating actions) writes a corresponding audit log entry before responding.

Request Lifecycle

1. The browser sends a request via an Axios instance configured with `withCredentials: true`, so the JWT cookie set at login is automatically attached.
2. Express applies middleware in order: CORS validation -> Helmet security headers -> `authenticateToken` (verifies the JWT) -> `authorizeRoles()` / `enforceBaseScope()` (RBAC).
3. The controller executes parameterized SQL against PostgreSQL.
4. For any mutation (purchase, transfer, assignment, expenditure), `auditService.writeAuditLog()` inserts a corresponding row into `audit_logs`.
5. The controller returns JSON; React updates component state and re-renders.

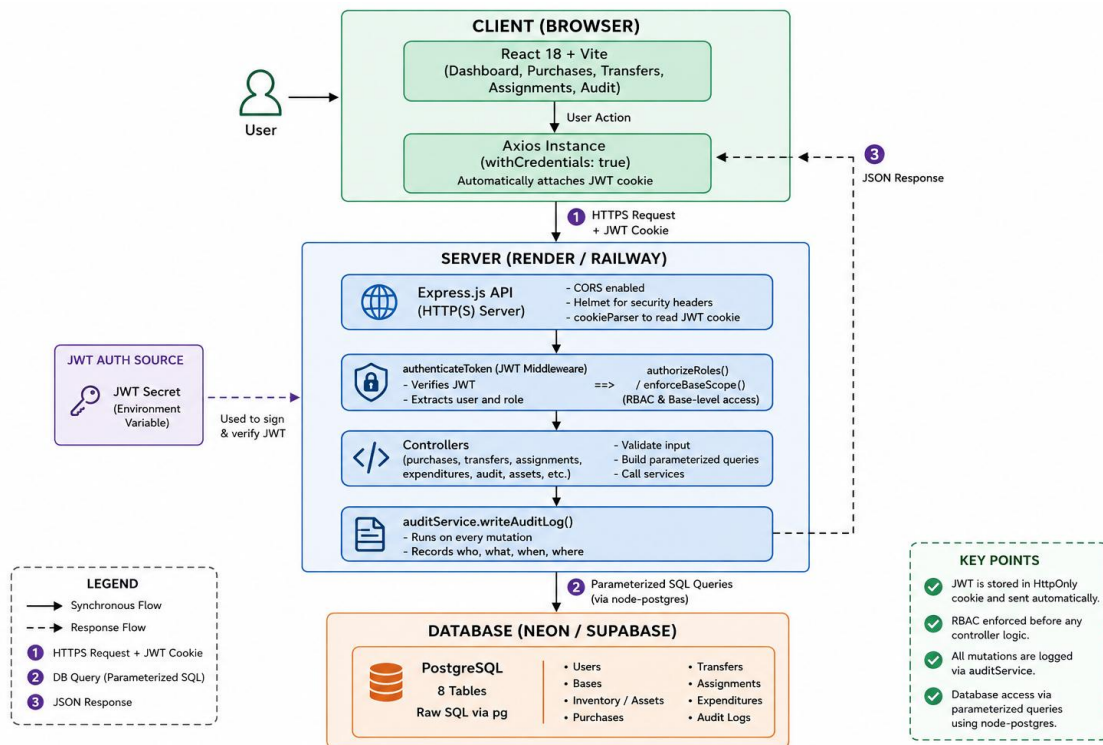


Figure 1 — Request flow across client, server, and database tiers

Production Cross-Origin Cookie Handling

Because the frontend (Vercel) and backend (Render) are deployed on different domains, the login cookie configuration adapts automatically: in production it is issued with `Secure: true` and `SameSite: None`; locally it falls back to `Secure: false` and `SameSite: Lax`. The backend also sets `app.set('trust proxy', 1)` so it can correctly detect HTTPS behind a reverse proxy.

3. Database Schema (ER Diagram)

The schema consists of 8 tables under PostgreSQL, with foreign-key constraints enforcing referential integrity and CHECK constraints preventing invalid roles, categories, or negative quantities at the database level.

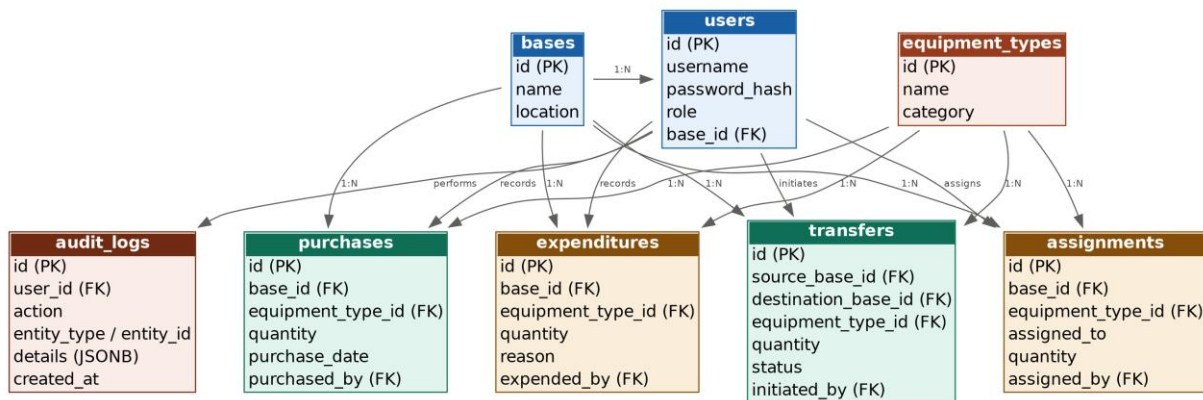


Figure 2 — Entity-relationship diagram

Table Reference

Table	Purpose	Key columns
bases	Registry of military bases	id, name, location
users	Auth credentials, role, base assignment	id, username, password_hash, role, base_id (FK)
equipment_types	Asset catalogue	id, name, category (WEAPON / VEHICLE / AMMUNITION)
purchases	Incoming stock events per base	base_id, equipment_type_id, quantity, purchase_date, purchased_by
transfers	Atomic cross-base movements	source_base_id, destination_base_id, equipment_type_id, quantity, status, initiated_by
assignments	Asset allocation to a unit/personnel (quantity-only)	base_id, equipment_type_id, assigned_to, quantity, assigned_by
expenditures	Consumed / spent assets	base_id, equipment_type_id, quantity, reason, expended_by
audit_logs	Immutable log of every mutation	user_id, action, entity_type, entity_id, details (JSONB), created_at

Constraints & Indexing

- `users.role` is CHECK-constrained to ADMIN / BASE_COMMANDER / LOGISTICS_OFFICER.
- `equipment_types.category` is CHECK-constrained to WEAPON / VEHICLE / AMMUNITION.
- `transfers.status` is CHECK-constrained to PENDING / IN_TRANSIT / COMPLETED.
- All quantity columns carry CHECK (`quantity > 0`).
- Indexes exist on every foreign key used in dashboard filtering (`base_id`, `equipment_type_id`, `source_base_id`, `destination_base_id`) plus `created_at`, to keep aggregation queries fast as data grows.

4. API Endpoint Reference

All routes are prefixed with `/api`. Authentication is via an `httpOnly` JWT cookie set on login (not a Bearer header) — the Axios client must be configured with `withCredentials: true` for protected calls to succeed.

Method	Path	Access	Description
POST	<code>/api/auth/login</code>	Public	Validates credentials, sets JWT cookie, returns user profile
POST	<code>/api/auth/logout</code>	Authenticated	Clears the session cookie
GET	<code>/api/auth/me</code>	Authenticated	Returns the current user's profile + base name
GET	<code>/api/assets/dashboard</code>	Authenticated	Aggregated Opening/Net Movement/Closing metrics. Filters: <code>baseId</code> , <code>equipmentTypeId</code> , <code>startDate</code> , <code>endDate</code>
GET	<code>/api/assets/stock</code>	Authenticated	Live available-stock check for a base + equipment type
GET	<code>/api/assets/inventory</code>	Admin, Commander	Per-base, per-equipment current stock breakdown
GET	<code>/api/bases</code>	Authenticated	List all bases (dropdown population)
GET	<code>/api/equipment-types</code>	Authenticated	List equipment types, optional <code>?category=</code> filter

Method	Path	Access	Description
POST	/api/purchases	Admin, Logistics	Create a purchase (adds stock to a base)
GET	/api/purchases	Authenticated (scoped)	Paginated purchase history with filters
POST	/api/transfers	Admin, Logistics	Create an atomic cross-base transfer (BEGIN/COMMIT, stock-checked)
GET	/api/transfers	Authenticated (scoped)	Paginated transfer history with filters
POST	/api/assignments	Admin, Commander	Assign equipment to a unit/personnel label (stock-checked)
GET	/api/assignments	Authenticated (scoped)	Paginated assignment history
POST	/api/expenditures	Admin, Commander	Record consumed/expended stock (stock-checked)
GET	/api/expenditures	Authenticated (scoped)	Paginated expenditure history
GET	/api/audit-logs	Admin only	Paginated audit trail, filterable by action/entityType/date
GET	/health	Public	Health check — returns { status: "ok", timestamp }

"Scoped" access note

Endpoints marked Authenticated (scoped) are reachable by any logged-in role, but the `enforceBaseScope` middleware silently overwrites the `baseId` query parameter for `BASE_COMMANDER` users with their own assigned base — so a commander physically cannot retrieve another base's records, even by manually editing the request.

Example — Create Transfer

POST /api/transfers

```
{
  "sourceBaseId": 1,
  "destinationBaseId": 2,
  "equipmentTypeId": 5,
  "quantity": 20,
  "notes": "Routine resupply"
}
```

The controller validates required fields, opens a DB transaction, re-checks available stock at the source base inside that transaction (preventing race conditions), inserts the transfer row, writes an audit log entry, then commits. Any failure triggers a full rollback — the transfer either happens completely or not at all.

5. RBAC Authorization Matrix

Three system-wide roles are defined, with authorization enforced through two complementary layers. The `authorizeRoles(...)` middleware explicitly restricts access by role, immediately rejecting unauthorized requests with an HTTP 403 response. In addition, `enforceBaseScope` applies transparent data-level restrictions for Base Commanders, automatically narrowing their database queries to records belonging only to their assigned base. This layered approach ensures both role-based access control and base-specific data isolation while allowing authorized users to access only the information appropriate to their responsibilities.

Endpoint / Resource	ADMIN	BASE_COMMANDER	LOGISTICS_OFFICER
Dashboard metrics	All bases	Own base only	All bases
Base inventory (/assets/inventory)	Yes	Own base only	No
Bases / Equipment types (read)	Yes	Yes	Yes
Purchases — create	Yes	No	Yes
Purchases — read	All bases	Own base only	All bases
Transfers — create	Yes	No	Yes
Transfers — read	All bases	Own base only	All bases
Assignments — create/read	Yes	Own base only	No
Expenditures — create/read	Yes	Own base only	No
Audit logs — read	Yes	No	No

Role Summary

Role	Scope	Primary responsibilities
ADMIN	Global — all bases	Full visibility and control across purchases, transfers, assignments, expenditures, and the audit trail
BASE_COMMANDER	Own assigned base only	Manages assignments and expenditures for their base; views (but cannot create) purchases/transfers
LOGISTICS_OFFICER	Global (operational)	Creates purchases and initiates transfers across any base; no access to assignments, expenditures, or audit logs

6. Setup & Deployment Guide

Prerequisites

- Node.js v18 or higher
- A PostgreSQL instance (Neon or Supabase free tier recommended)

1. Configure the Backend

```
cd backend
cp .env.example .env
```

Edit .env and set the connection string:

```
DATABASE_URL=postgres://USER:PASSWORD@HOST:5432/DATABASE?sslmode=require
```

2. Initialize the Database

Run `backend/sql/schema.sql` against the PostgreSQL instance (via the Neon/Supabase SQL editor), then seed demo data:

```
npm install
npm run seed
```

The seed script prints all login credentials to the console and is idempotent (safe to re-run — it truncates and reseeds).

3. Start the Backend

```
npm run dev
# -> Server listening on http://localhost:5000
# -> Health check: GET http://localhost:5000/health
```

4. Start the Frontend

```
cd ../frontend
npm install
npm run dev
# -> App running at http://localhost:5173
```

Environment Variables — Production

Service	Variable	Description
Backend (Render)	DATABASE_URL	PostgreSQL connection string
	CLIENT_URL	Deployed frontend URL, e.g. <code>https://miltrack.vercel.app</code> (no trailing slash)
	JWT_SECRET	Cryptographically random secret, 32+ characters
Frontend (Vercel)	VITE_API_BASE_URL	Deployed API URL, e.g. <code>https://miltrack-api.onrender.com/api</code>

7. Sample Test Accounts

These accounts are created by `backend/seed.js` and are ready to use immediately after seeding — no manual account creation required.

Role	Username	Password	Base Access
ADMIN	admin_user	AdminPass123!	All bases (global)
BASE_COMMANDER	commander_north	CommandPass123!	Northern Command Base only
LOGISTICS_OFFICER	logistics_officer	LogisticsPass123!	Global operations

The seeder also populates 3 bases (Northern, Western, Eastern Command), 6 equipment types across the WEAPON / VEHICLE / AMMUNITION categories, and sample purchases, transfers, assignments, and expenditures — so the Dashboard shows real, non-zero figures immediately after setup.